
RSA: Clave Pública y Firma Digital

Ejercicios de Parciales – ITBA Criptografía y Seguridad (72.44)

24 de junio de 2026

Índice

1. Teoría: RSA	2
1.1. Generación de claves (GenRSA)	2
1.2. ¿Por qué funciona? (Teorema de Euler)	2
1.3. Problemas del Textbook RSA	2
1.4. PKCS #1 v1.5: Padding aleatorio	2
1.5. Firma digital RSA	3
1.6. PKI y Certificados	3
1.7. Resumen de roles de las claves	3
2. Parcial 2015 – Ejercicio 4 (Primera parte)	4
3. Parcial 2015 – Ejercicio 4 (Segunda parte)	4
4. Parcial 2015 – Ejercicio 5	5
5. Parcial 2025 Q1 – Ejercicio 5b	6
6. Parcial 2025 Q2 – Ejercicio 5c	7
7. Resumen y Tabla de Ejercicios RSA	8

1. Teoría: RSA

1.1. Generación de claves (GenRSA)

GenRSA — Generación de claves RSA

1. Elegir p, q primos. Calcular $n = p \cdot q$ (el “módulo”).
2. $\varphi(n) = (p-1)(q-1)$ (función de Euler).
3. Elegir e tal que $1 < e < \varphi(n)$ y $\gcd(e, \varphi(n)) = 1$ (coprimo).
4. Calcular d tal que $e \cdot d \equiv 1 \pmod{\varphi(n)}$ (inverso modular, Euclides extendido).
5. **Clave pública:** $pk = \langle n, e \rangle$. **Clave privada:** $sk = \langle n, d \rangle$.

Cifrar: $c = m^e \pmod n$, Descifrar: $m = c^d \pmod n$.

1.2. ¿Por qué funciona? (Teorema de Euler)

Corrección de RSA (Euler/Fermat)

Como $d \equiv e^{-1} \pmod{\varphi(n)}$, existe k entero tal que $ed = 1 + k\varphi(n)$. Entonces:

$$c^d \pmod n = (m^e)^d \pmod n = m^{ed} \pmod n = m^{1+k\varphi(n)} \pmod n = m \cdot (m^{\varphi(n)})^k \pmod n.$$

Por el **teorema de Euler**: $m^{\varphi(n)} \equiv 1 \pmod n$ (cuando $\gcd(m, n) = 1$). Luego $c^d \equiv m \pmod n$. Cifrado y descifrado son funciones inversas con el par de claves correcto.

1.3. Problemas del Textbook RSA

Problemas del Textbook RSA (sin padding)

- **Es determinístico:** el mismo m con el mismo pk siempre da el mismo c . No es CPA-Secure (el atacante puede probar mensajes).
- **e y m pequeños:** si $m^e < n$, no hay reducción modular y se puede calcular $\sqrt[e]{c}$ directamente (raíz e -ésima entera).
- **Módulos repetidos:** si dos pares comparten el mismo n , pueden factorizarlo.
- **Ataque de texto plano elegido (CCA):** Textbook RSA es homomórfico ($E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2)$), lo que permite ataques sofisticados.

1.4. PKCS #1 v1.5: Padding aleatorio

PKCS #1 v1.5 — Making RSA Non-Deterministic

Sea k la longitud de n en bytes. Se construye el mensaje con padding:

$$m' = 00 \parallel 02 \parallel r \parallel 00 \parallel m$$

donde r son bytes aleatorios $\neq 0$ (longitud $k - |m| - 3$). Luego se cifra m' .

- **El padding aporta no-determinismo:** cada cifrado es diferente aunque el mensaje sea el mismo.

- Se cree que es CPA-Secure, pero no es CCA-Secure (existen ataques como Bleichenbacher).
- En la práctica, siempre usar padding (PKCS o OAEP). Nunca “textbook RSA”.

1.5. Firma digital RSA

RSA como esquema de firma

En cifrado: la clave **pública** cifra, la **privada** descifra.

En firma: la clave **privada** firma, la **pública** verifica. Las claves tienen roles diferentes.

Firma estándar (Textbook RSA sign):

- $\text{Sign}_{sk}(m) = m^d \bmod n = s$ (la firma).
- $\text{Vrfy}_{pk}(m, s) = [s^e \bmod n \stackrel{?}{=} m]$.

Firma con hash (RSA-FDH / Hashed RSA):

- $\text{Sign}_{sk}(m) = [H(m)]^d \bmod n = s$.
- $\text{Vrfy}_{pk}(m, s)$: calcular $s^e \bmod n$ y verificar que $= H(m)$.
- **Por qué hashear:** (1) RSA solo puede firmar mensajes del tamaño del módulo; (2) el hash evita ataques existenciales (falsificación sin conocer el mensaje).

1.6. PKI y Certificados

Infraestructura de Clave Pública (PKI)

El problema de la clave pública: ¿cómo sé que la clave pública que recibo realmente pertenece a quien dice ser?

Solución: Certificados digitales y Autoridades de Certificación (CA).

- Una CA firma un certificado que asocia identidad $\leftrightarrow$ clave pública.
- El certificado contiene: nombre del titular, clave pública, fecha de vencimiento, firma de la CA.
- Para confiar en un certificado, hay que confiar en la CA que lo firmó (cadena de confianza).

1.7. Resumen de roles de las claves

Operación	Clave que actúa	Clave que verifica/deshace
Cifrado	Pública (del destinatario)	Privada (del destinatario)
Firma digital	Privada (del firmante)	Pública (del firmante)

Trampa clásica de parcial

“Los sistemas de cifrado de clave pública utilizan una clave para cifrar y otra para firmar.”
FALSO. En cifrado: la pública cifra, la privada descifra. En firma: la privada firma, la

pública verifica. Las claves no están asignadas a una operación específica (cifrar vs. firmar) sino a un *rol* (quién las usa). Esta distinción es clave en los parciales.

2. Parcial 2015 – Ejercicio 4 (Primera parte)

Consigna

En un esquema de clave pública, seleccionar la opción correcta:

- (a) La clave pública se mantiene en secreto y la clave privada se distribuye autenticada.
- (b) La clave privada se mantiene en secreto y la clave pública se distribuye autenticada.
- (c) La clave privada se mantiene en secreto y la clave pública también.
- (d) La clave privada se mantiene en secreto y la clave pública se mantiene en un KDC.

Resolución. Respuesta: (b).

En un esquema de clave pública (asimétrico):

- La **clave privada** se mantiene **en secreto** — solo el dueño la conoce.
- La **clave pública** se **distribuye** libremente, pero debe ser **autenticada** (para evitar ataques de suplantación de identidad). Esto es exactamente el rol de los certificados digitales y la PKI.

Por qué las demás son falsas:

- (a) Invierte los roles: la pública es la que se distribuye.
- (c) Ambas en secreto sería un sistema simétrico, no asimétrico.
- (d) La clave pública en un KDC (Key Distribution Center) es el modelo de Kerberos (clave simétrica), no de clave pública.

Tips y Anotaciones

Patrón: cuando el enunciado pregunte sobre la distribución de claves en criptografía asimétrica, la respuesta es siempre: **privada = secreta, pública = distribuida autenticada**. La palabra “autenticada” es clave: si distribuyo la clave pública sin autenticar, un atacante puede hacer un ataque MitM suplantando la identidad.

3. Parcial 2015 – Ejercicio 4 (Segunda parte)

Consigna

Para reducir ciertos ataques en el esquema de firma RSA se sugiere usar $\pi = (\text{Gen}, \text{Sign}, \text{Vrfy})$ donde s (la firma) se calcula como $s := [H(m)]^d \bmod N$, siendo $(N, d) = sk$ y H una función de hash resistente a colisiones. Para verificar la firma, ¿cuál es el proceso correcto que debe seguir Vrfy?

- (a) Vrfy debe recibir s y efectuar $s^e \bmod N$ y $H(s^e \bmod N)$.
- (b) Vrfy debe recibir s y efectuar $s^{(-1)^e} \bmod N$.

- (c) Vrfy debe recibir m y s y efectuar $s^e \bmod N$ y comparar con $H(m)$.
- (d) Vrfy debe recibir m y s y efectuar $m^e \bmod N$ y comparar con $H(s)$.

Resolución. Respuesta: (c).

El proceso es:

1. **Firma (Sign):** $s = [H(m)]^d \bmod N$.
2. **Verificación (Vrfy):** dado (m, s) :
 - Calcular $s^e \bmod N$ — esto “deshace” la firma con la clave pública.
 - Calcular $H(m)$ — el hash del mensaje recibido.
 - Verificar que $s^e \bmod N \stackrel{?}{=} H(m)$.

Por qué las demás son falsas:

- (a) Solo recibe s , no m : no puede calcular $H(m)$ para comparar.
- (b) $s^{(-1)^e}$ no tiene sentido algebraico en este contexto.
- (d) m^e cifra el mensaje original, no está relacionado con la firma; $H(s)$ tampoco tiene sentido.

Tips y Anotaciones

Mnemotecnia para firma RSA con hash:

“Sign priva-da, Hash primero; Vrfy publi-ca, descifra y compara con Hash del mensaje.”

- **Sign:** toma $m \rightarrow$ hashea $\rightarrow$ eleva a la d (privada) $\rightarrow$ produce s .
- **Vrfy:** toma $(m, s) \rightarrow$ eleva s a la e (pública) $\rightarrow$ compara con $H(m)$.
- **Key insight:** $s^e = [H(m)^d]^e = H(m)^{de} = H(m)^{1+k\varphi(n)} = H(m)$ (por Euler). Por eso la comparación funciona.
- **Por qué hashear y no firmar m directamente:** (1) RSA solo firma mensajes $< n$; (2) sin hash, el adversario puede fabricar firmas válidas para mensajes que no eligió (ataque existencial).

4. Parcial 2015 – Ejercicio 5

Consigna

Protocolo de autenticación RSA. El siguiente protocolo de autenticación mutua usa certificados y firmas RSA:

1. $A \rightarrow B$: n_A (nonce de A)
2. $B \rightarrow A$: $\text{Cert}_B, n_B, \text{Sign}_B(n_A, n_B)$
3. $A \rightarrow B$: $\text{Cert}_A, \text{Sign}_A(n_B, n_A)$

Mostrar cómo un adversario C puede autenticarse como A ante B .

Resolución. Este protocolo tiene una vulnerabilidad de **ataque de reflexión** (o ataque de entrelazado). El adversario C puede autenticarse como A ante B sin conocer la clave privada de A :

Sesión 1 (C ataca a B fingiendo ser A):

1. $C \rightarrow B: n_C$ (C inicia, fingiendo ser A , con un nonce propio)
2. $B \rightarrow C: \text{Cert}_B, n_B, \text{Sign}_B(n_C, n_B)$

Ahora C necesita $\text{Sign}_A(n_B, n_C)$ para completar el paso 3. C abre una **sesión paralela**:

Sesión 2 (C ataca a B abriendo otra sesión):

1. $C \rightarrow B: n_B$ (C envía el nonce de B como su nonce inicial)
2. $B \rightarrow C: \text{Cert}_B, n_{B'}, \text{Sign}_B(n_B, n_{B'})$

Pero C necesita que B le devuelva $\text{Sign}_A(n_B, n_C)$. El ataque es posible si $A = B$ (sesión de reflexión) o si B es convencido de firmar lo necesario.

En la variante más directa (C se hace pasar por A ante B):

- C inicia sesión con B enviando n_A (copiando o eligiendo un nonce).
- B responde con $\text{Sign}_B(n_A, n_B)$: el desafío es n_B .
- C necesita $\text{Sign}_A(n_B, n_A)$. Si C puede inducir a A a firmar eso (por ejemplo, abriendo una sesión con A donde A firme el nonce n_B), C puede completar la autenticación.
- El protocolo **no incluye el nombre del destinatario dentro de lo firmado**, por lo que la misma firma puede usarse en diferentes contextos.

Defensa: incluir la identidad del destinatario en el mensaje firmado:

- Paso 2: $\text{Sign}_B(A, n_A, n_B)$ (incluir el nombre del interlocutor).
- Paso 3: $\text{Sign}_A(B, n_B, n_A)$ (idem).

Así una firma obtenida en una sesión no puede reutilizarse en otra.

Tips y Anotaciones

Análisis de protocolos de autenticación — checklist:

- **¿Los mensajes firmados incluyen la identidad del interlocutor?** Si no, son vulnerables a ataque de reflexión.
- **¿Los nonces son suficientemente aleatorios?** Si no, hay ataques de replay.
- **¿El protocolo autentica ambas partes?** Un protocolo unilateral no da autenticación mutua.
- **¿Los certificados están verificados?** Si el certificado no se valida, hay MitM.
- **Nonce = Number used once:** aleatorio, para demostrar “freshness” (que el mensaje es reciente, no un replay).
- La vulnerabilidad de reflexión es un clásico: el adversario abre dos sesiones en paralelo y usa la firma de una para completar la otra.

5. Parcial 2025 Q1 – Ejercicio 5b

Consigna

Verdadero o Falso (justificar): “Los sistemas de encriptación basados en clave pública utilizan una de las claves para encriptar/desencriptar y la otra para firmar.”

Resolución. FALSO.

La afirmación mezcla incorrectamente las operaciones. La correcta distinción es:

Operación	Usa clave	Verifica/deshace con
Cifrado	Pública (del destinatario)	Privada (del destinatario)
Descifrado	Privada (del destinatario)	–
Firma	Privada (del firmante)	–
Verificación de firma	–	Pública (del firmante)

No hay “una clave para cifrar/descifrar” y “otra para firmar”. Las claves tienen roles determinados por su naturaleza (pública vs. privada):

- La **clave privada** se usa para: descifrar mensajes dirigidos a uno mismo, y firmar mensajes propios.
- La **clave pública** se usa para: cifrar mensajes dirigidos al dueño, y verificar firmas del dueño.

Tips y Anotaciones

Este es un ejercicio de comprensión conceptual pura. El error del enunciado es pensar en las claves como “una para X y otra para Y”, cuando en realidad cada clave aparece en dos operaciones distintas (una como agente, otra como verificador). Recordar: *la clave que “actúa” siempre es la privada (descifrar, firmar); la que “verifica” siempre es la pública (cifrar para alguien, verificar su firma).*

6. Parcial 2025 Q2 – Ejercicio 5c

Consigna

Verdadero o Falso (justificar): “El uso de padding aleatorio en la implementación del algoritmo de clave pública de RSA es para que el algoritmo sea seguro a ataque de textos cifrados elegidos.”

Resolución. FALSO.

El padding aleatorio (PKCS #1 v1.5 y OAEP) convierte al RSA determinístico en **no-determinístico**, lo que lo hace **CPA-Secure** (seguro ante ataque de texto plano elegido), **no** CCA-Secure (seguro ante ataque de texto cifrado elegido).

La distinción:

- **CPA (Chosen Plaintext Attack):** el atacante elige plaintexts y ve sus cifrados. El RSA textbook (determinístico) falla porque el mismo m siempre da el mismo c . El padding aleatorio hace que el mismo m dé distintos $c \Rightarrow$ CPA-Secure.

- **CCA (Chosen Ciphertext Attack):** el atacante elige ciphertexts y ve sus descifrados. PKCS #1 v1.5 **no** protege contra CCA: existen ataques como el de **Bleichenbacher (1998)** que explotan el oráculo de descifrado para recuperar el texto plano.
- Para CCA-Security se necesita **OAEP** (Optimal Asymmetric Encryption Padding) u otras construcciones probablemente seguras bajo CCA.

Tips y Anotaciones

Jerarquía de seguridad que hay que recordar:

CCA-Secure $\Rightarrow$ CPA-Secure $\Rightarrow$ Secure against eavesdropping

- **RSA Textbook (sin padding):** solo seguro si el atacante no puede ver ningún cifrado (poco útil en la práctica).
- **RSA + PKCS #1 v1.5:** CPA-Secure (no determinístico), pero Bleichenbacher mostró que NO es CCA-Secure.
- **RSA + OAEP:** se demuestra CCA-Secure bajo suposiciones criptográficas estándar.
- **Trampa del enunciado:** la afirmación dice “CCA” pero el padding solo da “CPA”. Esta confusión entre CPA y CCA es uno de los errores más penalizados en parciales de criptografía.

7. Resumen y Tabla de Ejercicios RSA

Año	Ej.	Tema	Respuesta
2015	4a	Distribución de claves en PKI	(b): privada secreta, pública autenticada
2015	4b	Verificación de firma RSA con hash	(c): Vrfy recibe (m, s) , calcula s^e y $H(m)$
2015	5	Ataque a protocolo de autenticación RSA	Ataque de reflexión / entrelazado
2025 Q1	5b	Roles de las claves (V/F)	FALSO: claves no se asignan a operaciones
2025 Q2	5c	Padding y CPA/CCA (V/F)	FALSO: padding da CPA-Secure, no CCA-Secure

Trampas frecuentes en RSA

1. **CPA vs CCA:** el padding aleatorio hace RSA CPA-Secure. Para CCA-Secure se necesita OAEP u otras construcciones.
2. **Roles de las claves:** la privada descifra y firma; la pública cifra y verifica. No hay “una para cifrar/descifrar y otra para firmar”.
3. **Firma RSA hashed:** Sign calcula $[H(m)]^d$; Vrfy recibe (m, s) y compara s^e con $H(m)$.
4. **Verificación requiere el mensaje m :** en hashed RSA, Vrfy no puede funcionar solo con s (necesita m para calcular $H(m)$).
5. **Protocolo de autenticación:** si lo firmado no incluye la identidad del destinatario, hay vulnerabilidad de reflexión.
6. **e y d se usan en módulo $\varphi(n)$:** $e \cdot d \equiv 1 \pmod{\varphi(n)}$, no módulo n .